



Examen Transversal Final

API RESTful en PHP Puro para Gestion de Inventario B2B

Caso: TodoCamisetas

Asignatura:	Desarrollo Backend - IF201IINF
Instituto:	Instituto Profesional San Sebastian
Evaluacion:	Examen Transversal Final
Estudiante:	Rodrigo Alexis Soto Cifuentes
Docente:	Raul Bilbao
Fecha de entrega:	Septiembre 2026

INDICE

1. Introduccion
2. Indicador 1.1.1 - Arquitectura y Modelo de Datos
 - 2.1. Estructura de directorios
 - 2.2. Modelo de datos y relaciones
3. Indicador 1.2.1 - Enrutamiento y Validacion
 - 3.1. Tabla completa de rutas (26 endpoints)
 - 3.2. Validacion en controladores
4. Indicador 2.1.1 - Documentacion de Endpoints (OpenAPI)
5. Indicador 2.2.3 - Content-Type, cURL y Postman
6. Indicador 3.1.1 - CRUD y Relacion M:N de Tallas
7. Indicador 3.2 - Optimizacion, Transacciones y Precio Final
8. Evidencia de Pruebas Ejecutadas
9. Video Demostrativo
10. Paquete de Entregable
11. Conclusiones
12. Uso de Inteligencia Artificial
13. Referencias

1. INTRODUCCION

El presente informe corresponde al **Examen Transversal Final** de la asignatura Desarrollo Backend (IF201IINF) del Instituto Profesional San Sebastian. El objetivo es diseñar, construir y documentar una API RESTful en **PHP puro (sin frameworks)** para **TodoCamisetas**, proveedor mayorista de camisetas de futbol que opera bajo un modelo de negocio B2B con tiendas minoristas.

La API resuelve tres necesidades del caso: (1) gestion de inventario de camisetas, (2) gestion de clientes B2B con categorias Regular y Preferencial, y (3) calculo dinamico del **precio final** de un producto segun el cliente que realiza la consulta. Ademias, cada camiseta puede estar disponible en varias tallas con su propio stock, lo que exige modelar una relacion **muchos a muchos** entre camisetas y tallas.

La solucion fue desarrollada en **PHP 8.x puro**, sin frameworks, siguiendo una separacion de responsabilidades tipo MVC (modelos, controladores, core) con un enrutador propio basado en **expresiones regulares** que despacha a **metodos estaticos** de los controladores, tal como exige el enunciado. La persistencia se realiza en **MySQL/InnoDB** mediante **PDO** con sentencias preparadas y transacciones.

Los resultados de aprendizaje demostrados cubren: identificacion de componentes de arquitectura backend (1.1.1), disenio de solicitudes HTTP (1.2.1), disenio funcional de la API (2.1.1), implementacion de endpoints documentados (2.2.3), operaciones CRUD eficientes incluyendo la relacion M:N de tallas (3.1.1), y optimizacion del rendimiento con transacciones y la regla de negocio de precio final (3.2).

2. INDICADOR 1.1.1 - ARQUITECTURA Y MODELO DE DATOS

2.1. Estructura de directorios

El proyecto se organiza como front controller + MVC en PHP puro. La siguiente estructura resume los directorios y su funcion:

```

todocamisetas-api/
|-- config/
|   |-- database.php      Conexion PDO singleton (host, usuario, charset)
|-- app/
|   |-- core/
|   |   |-- Router.php    Enrutamiento por expresiones regulares
|   |   |-- Response.php  Salida JSON unificada + codigos HTTP
|   |-- models/
|   |   |-- Camiseta.php   CRUD + JOIN a clientes + regla de precio final
|   |   |-- Cliente.php   CRUD + validacion de RUT unico
|   |   |-- Talla.php     CRUD del catalogo de tallas
|   |   |-- CamisetaTalla.php Tabla pivote (relacion muchos a muchos)
|   |-- controllers/
|   |   |-- CamisetaController.php index, show, store, update, destroy
|   |   |-- ClienteController.php index, show, store, update, destroy, camisetas
|   |   |-- TallaController.php  CRUD catalogo + asignar/actualizarStock/quitar
|-- public/
|   |-- index.php         Front controller: unico punto de entrada
|   |-- .htaccess         Reescritura de URLs hacia index.php
|-- docs/
|   |-- openapi.yaml      Documentacion Swagger/OpenAPI
|   |-- todocamisetas.postman_collection.json Coleccion Postman (31 peticiones)
|   |-- curl_ejemplos.md  Ejemplos cURL de cada endpoint
|-- database.sql          Script de creacion + datos de ejemplo
|-- EVIDENCIA_PRUEBAS.md  Resultados reales de pruebas ejecutadas

```

Justificacion de cada componente:

public/index.php (Front Controller)

Unico punto de entrada de toda la API. Centraliza las cabeceras CORS, el manejo de OPTIONS (preflight), la carga de dependencias y el registro de las 26 rutas. Evita repetir esta logica en cada archivo PHP.

app/core/Router.php

Compara la URI normalizada contra expresiones regulares registradas y, si hay coincidencia de metodo HTTP, invoca el metodo estatico del controlador correspondiente via call_user_func. Distingue 404 (ninguna ruta calza) de 405 (la ruta existe pero el verbo HTTP no corresponde, devolviendo el header Allow).

app/core/Response.php

Punto unico de salida: garantiza que toda respuesta incluya Content-Type: application/json, fija el codigo HTTP y serializa el cuerpo en JSON legible (JSON_PRETTY_PRINT, JSON_UNESCAPED_UNICODE). Tambien centraliza la lectura y validacion del body JSON entrante.

app/models/*.php

Unico lugar donde existe SQL. Cada modelo expone metodos estaticos (all, find, create, update, delete, existe) que usan PDO con sentencias preparadas. Se eligio PDO en vez de mysqli por su soporte nativo de sentencias preparadas con parametros nombrados, su API orientada a objetos y su compatibilidad con transacciones explicitas (beginTransaction/commit/rollback), necesarias para las operaciones compuestas del examen.

app/controllers/*.php

Validan la entrada del request, aplican las reglas de negocio (por ejemplo, no eliminar un cliente con camisetas asignadas) y deciden el codigo HTTP de respuesta. No contienen SQL: delegan toda persistencia a los modelos, manteniendo una separacion de responsabilidades clara pese a no usar un framework.

config/database.php

Crea y reutiliza una unica conexion PDO por request (patron singleton), con PDO::ATTR_EMULATE_PREPARES en false para que las sentencias preparadas se ejecuten realmente en el servidor MySQL y no sean emuladas por el driver.

2.2. Modelo de datos y relaciones

El modelo cubre cuatro tablas en MySQL/InnoDB. Se opto por una **relacion directa 1:N** (cliente_id en camisetas) para representar el 'dueno' comercial de cada producto, y una **relacion N:N** mediante tabla pivote para camisetas y tallas, ya que una camiseta puede tener varias tallas y una talla es reutilizada por muchas camisetas.

Tabla	Campos clave	Relacion
clientes	id, nombre_comercial, rut (UNIQUE), categoria ENUM('Regular', 'Premium'), precio_oferta CHECK 0-100	1 cliente a N camisetas
camisetas	id, titulo, club, pais, tipo, color, precio, precio_oferta (NULL), codigo_producto (UNIQUE), cliente_id (FK, NULL)	N camisetas <-> N tallas
tallas	id, codigo (UNIQUE), descripcion	Catalogo reutilizable, N <-> N con camisetas
camiseta_tallas	camiseta_id (FK), talla_id (FK), stock, PK compuesta (camiseta_id, talla_id)	Tabla pivote: resuelve N:N y guarda el stock por combinacion

La clave primaria compuesta (**camiseta_id, talla_id**) en camiseta_tallas impide asignar dos veces la misma talla al mismo producto directamente a nivel de base de datos. Ambas claves foraneas usan **ON DELETE CASCADE**: si se elimina una camiseta o una talla, sus asignaciones en la tabla pivote se eliminan automaticamente, evitando filas huérfanas. En cambio, camisetas.cliente_id usa **ON DELETE RESTRICT**: la base de datos impide fisicamente eliminar un cliente que aun tiene productos asignados, reforzando a nivel de esquema la regla de negocio pedida en la Tarea 5.

Diagrama de arquitectura y diagrama entidad-relacion disponibles como imagenes en docs/diagrama_arquitectura.png y docs/modelo_datos.png, adjuntos en el paquete de entrega.

3. INDICADOR 1.2.1 - ENRUTAMIENTO Y VALIDACION

3.1. Tabla completa de rutas

El enrutador registra **26 rutas** mediante expresiones regulares con grupos nombrados (?P<id>) que se convierten directamente en los parametros del controlador. Las rutas anidadas (tallas de una camiseta) se registran antes que las rutas generales de camiseta por id, ya que el Router devuelve la primera coincidencia. Verificado en vivo contra el servidor: GET /api devuelve esta misma tabla.

Método	Expresión regular	Controlador	Propósito
GET	#^/api/camisetas/(?P\d+)/tallas\$#	TallaController@deCamiseta	Lista tallas y stock de una camiseta
POST	#^/api/camisetas/(?P\d+)/tallas\$#	TallaController@asignar	Asigna una talla del catalogo a una camiseta
PUT	#^/api/camisetas/(?P\d+)/tallas/(?P\d+)\$#	TallaController@actualizarStock	Actualiza el stock de una talla
PATCH	#^/api/camisetas/(?P\d+)/tallas/(?P\d+)\$#	TallaController@actualizarStock	Actualiza parcialmente el stock
DELETE	#^/api/camisetas/(?P\d+)/tallas/(?P\d+)\$#	TallaController@quitar	Quita una talla de una camiseta
GET	#^/api/camisetas\$#	CamisetaController@index	Lista camisetas (filtros club, tipo, cliente_id + paginacion)
GET	#^/api/camisetas/(?P\d+)\$#	CamisetaController@show	Camiseta con tallas y precio_final segun cliente_id
POST	#^/api/camisetas\$#	CamisetaController@store	Crea camiseta, opcionalmente con tallas
PUT	#^/api/camisetas/(?P\d+)\$#	CamisetaController@update	Actualiza completo
PATCH	#^/api/camisetas/(?P\d+)\$#	CamisetaController@update	Actualiza parcial
DELETE	#^/api/camisetas/(?P\d+)\$#	CamisetaController@destroy	Elimina camiseta (cascada a tallas)
GET	#^/api/clientes/(?P\d+)/camisetas\$#	ClienteController@camisetas	Camisetas de un cliente con precio_final
GET	#^/api/clientes\$#	ClienteController@index	Lista clientes (filtro categoria)
GET	#^/api/clientes/(?P\d+)\$#	ClienteController@show	Obtiene un cliente
POST	#^/api/clientes\$#	ClienteController@store	Crea cliente B2B
PUT	#^/api/clientes/(?P\d+)\$#	ClienteController@update	Actualiza completo
PATCH	#^/api/clientes/(?P\d+)\$#	ClienteController@update	Actualiza parcial
DELETE	#^/api/clientes/(?P\d+)\$#	ClienteController@destroy	Elimina si no tiene camisetas (409 si tiene)
GET	#^/api/tallas\$#	TallaController@index	Lista catalogo de tallas
GET	#^/api/tallas/(?P\d+)\$#	TallaController@show	Obtiene una talla
POST	#^/api/tallas\$#	TallaController@store	Crea talla en catalogo
PUT	#^/api/tallas/(?P\d+)\$#	TallaController@update	Actualiza completo
PATCH	#^/api/tallas/(?P\d+)\$#	TallaController@update	Actualiza parcial
DELETE	#^/api/tallas/(?P\d+)\$#	TallaController@destroy	Elimina talla (cascada)
GET	#^/\$#	ApiController@bienvenida	Informacion general de la API
GET	#^/api\$#	ApiController@bienvenida	Informacion general + tabla de rutas

3.2. Validacion en controladores

Cada metodo estatico store()/update() valida los campos obligatorios antes de tocar la base de datos, y responde 422 con la lista de errores si algo falta. Ejemplo en CamisetaController:

```
private static function validar(array $cuerpo, bool $exigirTodos): array
{
    $errores = [];
    if ($exigirTodos) {
        foreach (['titulo','club','pais','tipo','color','precio','codigo_producto'] as $campo) {
            if (!isset($cuerpo[$campo]) || trim((string)$cuerpo[$campo]) === '') {
                $errores[] = "El campo \"$campo\" es obligatorio.";
            }
        }
    }
    if (isset($cuerpo['precio']) && (!is_numeric($cuerpo['precio']) || $cuerpo['precio'] <= 0)) {
        $errores[] = 'El campo "precio" debe ser un numero mayor que cero.';
    }
    if (isset($cuerpo['precio_oferta']) && $cuerpo['precio_oferta'] !== null) {
        if (!is_numeric($cuerpo['precio_oferta']) || $cuerpo['precio_oferta'] <= 0) {
            $errores[] = 'El campo "precio_oferta" debe ser un numero mayor que cero.';
        } elseif (isset($cuerpo['precio']) && (float)$cuerpo['precio_oferta'] >=
            (float)$cuerpo['precio']) {
            $errores[] = 'El campo "precio_oferta" debe ser menor que el precio base.';
        }
    }
    return $errores;
}
```

El router distingue explícitamente 404 (ruta inexistente) de 405 (ruta existente, verbo no soportado), devolviendo en este último caso el header **Allow** con los métodos válidos. Verificado en vivo: DELETE /api/clientes responde 405 con Allow: GET, POST.

4. INDICADOR 2.1.1 - DOCUMENTACION DE ENDPOINTS (OPENAPI)

Se genero documentacion completa en formato **OpenAPI 3.0.3** (docs/openapi.yaml, 9 paths, 22 schemas, 24 operaciones), importable directamente en Swagger Editor. Cada operacion documenta metodo HTTP, ruta exacta con parametros, query params, cuerpo esperado (JSON), respuesta minima y todos los codigos de error posibles. A continuacion se detallan los endpoints centrales del caso de negocio:

Obtener camiseta con precio final

Campo	Detalle
Metodo / Ruta	GET /api/camisetas/{id}?cliente_id={cliente_id}
Descripcion	Devuelve una camiseta con el precio_final calculado segun el cliente indicado en la query. Si no se envia cliente_id, precio_final = precio base.
Request	Query param opcional: cliente_id (int)
Response 2xx	{"status":"success","code":200,"data":{"id":1,"precio":45000,"precio_oferta":38000,"precio_final":38000,"precio_detalle":{"tipo_cliente":"Preferencial","origen":"precio_oferta","descuento":7000}}}
Errores	404 si el id no existe; 404 si cliente_id no existe

Crear camiseta (con tallas opcionales)

Campo	Detalle
Metodo / Ruta	POST /api/camisetas
Descripcion	Crea una camiseta y, si se envia el arreglo tallas, asigna cada talla con su stock en la misma transaccion.
Request	{"titulo":"...", "club":"...", "pais":"Chile", "tipo":"Local", "color":"Rojo", "precio":45000, "codigo_producto":"SCL2 025L", "tallas":[{"talla_id":1, "stock":20}]}
Response 2xx	{"status":"success","code":201,"message":"Camiseta creada correctamente.", "data":{"id":8,...}}
Errores	422 campos faltantes o precio_oferta>=precio; 409 codigo_producto duplicado

Asignar talla a camiseta

Campo	Detalle
Metodo / Ruta	POST /api/camisetas/{id}/tallas
Descripcion	Asigna una talla del catalogo a una camiseta existente con su stock inicial.
Request	{"talla_id":3, "stock":20}
Response 2xx	{"status":"success","code":201,"message":"Talla asignada correctamente a la camiseta.", "data":{"talla_id":3, "codigo":"L", "stock":20}}
Errores	404 camiseta o talla inexistente; 409 talla ya asignada (PK compuesta); 422 stock invalido

Eliminar cliente

Campo	Detalle
Metodo / Ruta	DELETE /api/clientes/{id}
Descripcion	Elimina un cliente B2B solo si no tiene camisetas asignadas.
Request	Sin body
Response 2xx	{"status":"success","code":200,"message":"Cliente con ID 4 eliminado correctamente."}
Errores	404 cliente inexistente; 409 si tiene camisetas asignadas (incluye camisetas_asociadas: N)

Todos los modelos siguen la firma requerida por el enunciado (all(), find(), create(), update(), delete(), existe()), con sentencias preparadas y manejo de excepciones PDOException capturadas en cada controlador y traducidas a codigos HTTP claros (400, 404, 409, 422, 500).

5. INDICADOR 2.2.3 - CONTENT-TYPE, cURL Y POSTMAN

La clase Response.php es el **unico punto de salida** de toda la API: toda respuesta, exitosa o de error, pasa por Response::exito() o Response::error(), que fijan siempre la cabecera Content-Type: application/json antes de imprimir el cuerpo. Esto se verifico en vivo en las mas de 30 peticiones de prueba realizadas (ver seccion 8): ninguna respuesta devolvio HTML o texto plano.

Ejemplos cURL (incluyendo tallas, el recurso con relacion M:N):

```
curl -X GET "http://localhost/todocamisetas-api/public/api/camisetas/1?cliente_id=1"

curl -X POST "http://localhost/todocamisetas-api/public/api/camisetas/1/tallas" \
-H "Content-Type: application/json" \
-d '{"talla_id":3,"stock":20}'

curl -X PATCH "http://localhost/todocamisetas-api/public/api/camisetas/1/tallas/3" \
-H "Content-Type: application/json" \
-d '{"stock":15}'

curl -X DELETE "http://localhost/todocamisetas-api/public/api/camisetas/1/tallas/3"

curl -X DELETE "http://localhost/todocamisetas-api/public/api/clientes/1"
# 409: No se puede eliminar el cliente porque tiene 3 camiseta(s) asignada(s).
```

El listado completo (todos los endpoints, incluyendo casos de error) esta documentado en docs/curl_ejemplos.md, entregado junto al codigo fuente.

Coleccion Postman

Se genero programaticamente (para garantizar consistencia) una coleccion Postman v2.1.0 con **31 peticiones** organizadas en 5 carpetas (Camisetas, Clientes, Tallas - catalogo, Tallas - relacion M:N, Casos de error) y **36 ejemplos de respuesta guardados** (exito y error: 404, 409, 422). Incluye variables base_url, cliente_preferencial y cliente_regular para probar la regla de precio final sin editar cada request manualmente.

6. INDICADOR 3.1.1 - CRUD Y RELACION M:N DE TALLAS

La tabla camiseta_tallas resuelve la relacion muchos a muchos entre camisetas y tallas, con clave primaria compuesta y ambas claves foraneas ON DELETE CASCADE (ver definicion completa en la seccion 2.2). El controlador TallaController implementa el grupo completo de endpoints para esta relacion:

Metodo	Ruta	Validaciones antes de operar
GET	/api/camisetas/{id}/tallas	Verifica que la camiseta exista (404 si no)
POST	/api/camisetas/{id}/tallas	Camiseta existe -> talla existe en catalogo -> no duplicada (409) -> stock v
PUT / PATCH	/api/camisetas/{id}/tallas/{tallaId}	Camiseta existe -> asignacion existe (404) -> stock numerico >= 0
DELETE	/api/camisetas/{id}/tallas/{tallaId}	Camiseta existe -> asignacion existe (404) -> elimina

Modelo CamisetaTalla.php — metodo existe(), usado antes de asignar (409) y antes de actualizar/eliminar (404):

```
public static function existe(int $camisetaId, int $tallaId): bool
{
    $db = getDbConnection();
    $sentencia = $db->prepare(
        'SELECT COUNT(*) FROM camiseta_tallas
        WHERE camiseta_id = :camiseta_id AND talla_id = :talla_id'
    );
    $sentencia->execute([':camiseta_id' => $camisetaId, ':talla_id' => $tallaId]);
    return (int) $sentencia->fetchColumn() > 0;
}
```

Todas las escrituras (create, update, delete) de CamisetaTalla se envuelven en una transaccion explicita (beginTransaction/commit/rollBack), y las excepciones PDOException con codigo SQLSTATE 23000 (violacion de restriccion) se traducen a HTTP 409 en lugar de un 500 generico, mediante un metodo manejarErrorBD() reutilizado en los tres controladores.

Prueba en vivo (ver seccion 8): intentar asignar una talla ya asignada devuelve 409 respetando la clave primaria compuesta; intentar operar sobre una talla no asignada devuelve 404 con mensaje explicito.

7. INDICADOR 3.2 - OPTIMIZACION, TRANSACCIONES Y PRECIO FINAL

Regla de negocio: precio final

El enunciado plantea dos mecanismos de descuento que pueden coexistir: un precio_oferta por producto y un porcentaje_oferta por cliente. Se definio un orden de precedencia explicito, implementado en Camiseta::calcularPrecioFinal():

```
public static function calcularPrecioFinal(float $precio, ?float $precioOferta, ?array $cliente): array
{
    if ($cliente === null) {
        return ['precio_final' => round($precio, 2), 'origen' => 'precio_base', 'descuento' => 0.0];
    }
    if ($cliente['categoria'] === 'Preferencial' && $precioOferta !== null) {
        return ['precio_final' => round($precioOferta, 2), 'origen' => 'precio_oferta',
            'descuento' => round($precio - $precioOferta, 2)];
    }
    $porcentaje = (float) ($cliente['porcentaje_oferta'] ?? 0);
    if ($porcentaje > 0) {
        $descuento = $precio * ($porcentaje / 100);
        return ['precio_final' => round($precio - $descuento, 2), 'origen' => 'porcentaje_cliente',
            'descuento' => round($descuento, 2)];
    }
    return ['precio_final' => round($precio, 2), 'origen' => 'precio_base', 'descuento' => 0.0];
}
```

Verificacion en vivo contra la API real (servidor XAMPP, base todocamisetas), sobre la camiseta 1 (precio 45.000, precio_oferta 38.000):

Peticion	precio_final obtenido	Regla aplicada	Correcto
GET /camisetas/1 (sin cliente_id)	45.000	3 - precio base	Si
GET /camisetas/1?cliente_id=1 (90minutos, Preferencial)	38.000 (desc. 7.000)	1 - precio_oferta	Si
GET /camisetas/1?cliente_id=2 (tdeportes, Regular, 0%)	45.000	3 - precio base	Si
GET /camisetas/2?cliente_id=3 (GolStore, Regular, 5%)	42.750 (desc. 2.250)	2 - porcentaje_cliente	Si

Optimizacion: prevencion de N+1

Al listar camisetas (GET /api/camisetas), en vez de ejecutar una consulta de tallas por cada camiseta devuelta (problema N+1: con 50 productos serian 51 consultas), CamisetaTalla::deVariasCamisetas() trae las tallas de **todas** las camisetas de la pagina en **una sola consulta** con IN (...) y las agrupa en memoria por camiseta_id. Ademias, la consulta base de camisetas usa un **LEFT JOIN** a clientes para traer el dueño del producto sin una consulta adicional por fila.

Transacciones

Las operaciones compuestas usan transacciones explicitas: al crear una camiseta junto con su arreglo de tallas, si falla la insercion de alguna talla se revierte tambien la insercion de la camiseta (rollback), evitando productos huérfanos sin tallas validas. Lo mismo aplica a create/update/delete

en Cliente, Camiseta, Talla y CamisetaTalla.

8. EVIDENCIA DE PRUEBAS EJECUTADAS

Las pruebas se ejecutaron **en vivo** contra el servidor XAMPP local (Apache + MySQL), con la base todocamisetas importada desde database.sql, usando fetch() desde la consola del navegador. El detalle completo, incluyendo una incidencia transitoria detectada y resuelta, esta en EVIDENCIA_PRUEBAS.md. Resumen de los casos mas relevantes:

Campo	Detalle
Caso	Importacion de la BD
Endpoint	-
Entrada / condicion	Conteos esperados: 3 clientes, 7 camisetas, 6 tallas, 21 camiseta_tallas
Resultado obtenido	Coinciden exactamente. PASS.
Campo	Detalle
Caso	Precio final - 4 escenarios
Endpoint	GET /camisetas/{id}?cliente_id=
Entrada / condicion	Ver tabla seccion 7
Resultado obtenido	Los 4 escenarios correctos. PASS.
Campo	Detalle
Caso	Crear camiseta con tallas
Endpoint	POST /camisetas
Entrada / condicion	tallas:[{talla_id,stock}]
Resultado obtenido	201, camiseta + 2 tallas creadas atomicamente. PASS.
Campo	Detalle
Caso	Actualizar cliente completo
Endpoint	PUT /clientes/{id}
Entrada / condicion	Body con todos los campos
Resultado obtenido	200, cliente actualizado. PASS.
Campo	Detalle
Caso	Asignar talla duplicada
Endpoint	POST /camisetas/{id}/tallas
Entrada / condicion	talla_id ya asignada
Resultado obtenido	409, mensaje claro (PK compuesta respetada). PASS.
Campo	Detalle
Caso	Actualizar y quitar talla
Endpoint	PATCH y DELETE /camisetas/{id}/tallas/{tallaId}
Entrada / condicion	6 ciclos consecutivos asignar/eliminar
Resultado obtenido	200 en los 6 ciclos. PASS.
Campo	Detalle
Caso	JSON malformado
Endpoint	POST /clientes
Entrada / condicion	Body: {invalid json
Resultado obtenido	400, mensaje de sintaxis. PASS.

Campo	Detalle
Caso	Cliente inexistente
Endpoint	GET /clientes/9999
Entrada / condicion	-
Resultado obtenido	404. PASS.
Campo	Detalle
Caso	Metodo no permitido
Endpoint	DELETE /clientes
Entrada / condicion	-
Resultado obtenido	405, header Allow: GET, POST. PASS.
Campo	Detalle
Caso	Ruta inexistente
Endpoint	GET /no-existe
Entrada / condicion	-
Resultado obtenido	404, mensaje con metodo y ruta. PASS.
Campo	Detalle
Caso	Eliminar cliente con camisetas
Endpoint	DELETE /clientes/1
Entrada / condicion	Cliente 1 tiene 3 camisetas
Resultado obtenido	409, camisetas_asociadas: 3. PASS.
Campo	Detalle
Caso	Codigo de producto duplicado
Endpoint	POST /camisetas
Entrada / condicion	codigo_producto existente
Resultado obtenido	409. PASS.
Campo	Detalle
Caso	precio_oferta mayor que precio
Endpoint	POST /camisetas
Entrada / condicion	precio_oferta >= precio
Resultado obtenido	422, mensaje de validacion. PASS.

Todos los datos de prueba creados durante la sesion (cliente y camiseta de prueba) fueron eliminados al finalizar, restaurando el conteo original de 3 clientes y 7 camisetas.

9. VIDEO DEMOSTRATIVO

Se elaboro un guion detallado (GUION_VIDEO.md, incluido en el paquete) con escaleta por bloques de tiempo, mapeando cada bloque a los indicadores de la rubrica que demuestra. El video cubre: presentacion, arquitectura de carpetas, tabla de rutas, CRUD de camisetas y clientes en Postman, relacion M:N de tallas (incluyendo el caso 409 de duplicado), la regla de precio final con los dos clientes del caso, manejo de errores (400/404/405/409/422) y cierre con el repositorio del codigo.

Campo	Detalle
Herramienta	OBS Studio
Duracion estimada	8 a 12 minutos
Formato	MP4, 1920x1080, 16:9
Guion	GUION_VIDEO.md (9 bloques cronometrados)
Link del video	[COMPLETAR TRAS GRABAR Y SUBIR A LA PLATAFORMA INDICADA]

10. PAQUETE DE ENTREGABLE

El proyecto se entrega comprimido en un unico archivo .zip que incluye codigo fuente completo, este informe y el video (o su enlace, segun corresponda), organizados de forma clara:

```
examen_rodrigosoto.zip
|-- app/                (models, controllers, core)
|-- config/
|-- public/
|-- docs/              (openapi.yaml, coleccion Postman, curl_ejemplos.md,
|                       diagramas de arquitectura y modelo de datos)
|-- database.sql
|-- EVIDENCIA_PRUEBAS.md
|-- README.md
|-- GUION_VIDEO.md
`-- informe_examen_rodrigosoto.pdf  (este documento)
```

El codigo fuente esta ordenado siguiendo la separacion models/controllers/core descrita en la seccion 2, sin archivos sueltos ni carpetas vacias, listo para copiarse directamente a httdocs y ejecutarse tras importar database.sql.

11. CONCLUSIONES

Se desarrollo exitosamente una API RESTful en PHP puro para TodoCamisetas, cubriendo CRUD completo de camisetas y clientes, la relacion muchos a muchos con tallas y stock, y el calculo dinamico de precio final segun el cliente consultante, verificado en vivo contra los cuatro escenarios del caso de negocio.

El enrutamiento por expresiones regulares con metodos estaticos demostro ser suficiente para separar responsabilidades sin depender de un framework, y la distincion explicita entre 404 y 405 entrega informacion mas util al consumidor de la API que un enfoque generico.

La combinacion de restricciones a nivel de base de datos (claves foraneas con RESTRICT/CASCADE, UNIQUE, CHECK) con validacion a nivel de aplicacion crea una red de seguridad en dos capas: la aplicacion entrega mensajes claros y la base de datos garantiza integridad incluso ante errores de programacion.

Como mejora futura se recomienda: autenticacion por API key o JWT para diferenciar clientes que consultan la API, cache del catalogo de tallas (cambia con poca frecuencia), y pruebas automatizadas con PHPUnit en lugar de pruebas manuales via fetch/Postman.

12. USO DE INTELIGENCIA ARTIFICIAL

En la elaboracion del codigo y de este informe se utilizo **Claude** (Anthropic), un modelo de lenguaje de gran escala (LLM) basado en inteligencia artificial generativa.

1. Generacion de codigo PHP puro con patrones de diseno: Claude asistio en la implementacion del Router basado en expresiones regulares, la clase Response para centralizar la salida JSON, y los modelos/controladores siguiendo la firma requerida por el enunciado (all, find, create, update, delete), reduciendo tiempo de scaffolding y garantizando consistencia entre los tres recursos (camisetas, clientes, tallas).

2. Diseno de la regla de precio final: el enunciado planteaba dos mecanismos de descuento (precio_oferta por producto y porcentaje_oferta por cliente) sin especificar su precedencia cuando ambos aplican; se discutio con la IA y se definio explicitamente el orden de reglas 1-2-3 documentado en la seccion 7, decision que es responsabilidad del estudiante.

3. Documentacion y verificacion: Claude genero la documentacion OpenAPI, la coleccion Postman (via script, para asegurar consistencia entre las 31 peticiones) y ejecuto las pruebas en vivo reportadas en la seccion 8 directamente contra el servidor, incluyendo la deteccion, investigacion y documentacion honesta de una incidencia transitoria de conexion a MySQL.

Declaracion de uso de herramientas de Inteligencia Artificial: El presente informe fue desarrollado con apoyo de herramientas de Inteligencia Artificial (Claude, de Anthropic) como asistencia en la redaccion, estructuracion del contenido y generacion de codigo base. Todo el analisis, reflexion y comprension de los temas es responsabilidad del estudiante.

13. REFERENCIAS

- Anthropic. (2026). *Claude - Asistente de inteligencia artificial*. <https://www.anthropic.com>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation). University of California, Irvine.
- Mozilla Developer Network. (2024). *HTTP response status codes*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- PHP Group. (2024). *PHP 8.x documentation - PDO*. <https://www.php.net/manual/es/book.pdo.php>
- Postman Inc. (2024). *Postman API Platform documentation*. <https://learning.postman.com/docs/>
- Richardson, L., & Ruby, S. (2007). *RESTful web services*. O'Reilly Media.
- Swagger / SmartBear. (2024). *OpenAPI Specification 3.0.3*. <https://swagger.io/specification/>